

CHAPTER 15

Recursive attitude estimation

15.1. Introduction

This chapter describes how to design recursive attitude estimators. Attitude estimators are algorithms that can be used to determine the orientation of a spacecraft from sensor data.

There are two types of recursive estimators, batch and sequential. Batch estimators are used when you have collected a large span of data and want to process it at once. Sequential estimators are used when you want to use the data as it is collected.

15.2. Batch methods

Batch methods of attitude determination take a set of measurements over multiple time samples and attempt to fit a solution. There are four types of batch least-squares estimators.

1. least squares;
2. weighted least squares;
3. maximum likelihood;
4. Bayesian.

Given the measurement equation

$$z = Hx + v \quad (15.1)$$

where v is unbiased white noise, the least-squares estimator is

$$\hat{x} = (H^T H)^{-1} H^T z \quad (15.2)$$

where z may be a vector with measurements from different types of sensors taken at different times. The assumption is that the state being measured has not changed much during the interval. This process treats all measurements equally.

The next type of estimator is the weighted least squares with a weighting matrix R

$$\hat{x} = (H^T R^{-1} H)^{-1} H^T R^{-1} z \quad (15.3)$$

R may be chosen by any means desired. For example, if there are too many measurements from one sensor, rather than removing them completely one might deweight them.

The third kind of estimator is the maximum likelihood estimator. It is

$$\hat{x} = (H^T R^{-1} H)^{-1} H^T R^{-1} z \quad (15.4)$$

which is identical in form to the weighted least squares. The difference is that R is the covariance matrix for z . Thus, the selection of the weighting matrix is put on a firm statistical footing.

The final type of estimator is the Bayesian estimator which is

$$\hat{x} = (P_0^{-1} H^T R^{-1} H)^{-1} H^T R^{-1} z \quad (15.5)$$

where P_0 is the a priori covariance matrix for x . This assumes that we know something about x prior to taking any measurements.

It is instructive to look at the case of three sequential measurements. In this case, the measurement equation is

$$z = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \quad (15.6)$$

The least-squares estimator becomes

$$\hat{x} = \frac{1}{3}(z_1 + z_2 + z_3) \quad (15.7)$$

which is just the average of the three measurements. The weighted least-squares (and maximum likelihood) estimators become

$$\hat{x} = \frac{\left(\frac{z_1}{r_1} + \frac{z_2}{r_2} + \frac{z_3}{r_3} \right)}{\frac{1}{r_1} + \frac{1}{r_2} + \frac{1}{r_3}} \quad (15.8)$$

If the measurements happen to be identical, the weights have no affect on the estimate. Otherwise, the measurement with the smallest weight will be favored. The Bayesian estimator is

$$\hat{x} = \frac{\left(\frac{z_1}{r_1} + \frac{z_2}{r_2} + \frac{z_3}{r_3} \right)}{\frac{1}{r_1} + \frac{1}{r_2} + \frac{1}{r_3} + \frac{1}{p_0}} \quad (15.9)$$

In the limit, as p_0 goes to infinity, the estimate goes to zero.

The Differential Corrector algorithm is

$$x_{k+1} = x_k + [S_0 + h_k^T W H_k]^{-1} [H_k^T W (\gamma - h_k) + S_0(x_0 - x_k)] \quad (15.10)$$

where W is the measurement weighting matrix and S_0 is the a priori covariance matrix of vector x_0 . For the purposes of evaluating the quality of the result the condition number of the inverse should be evaluated. S_0 tends to improve the conditioning.

The conjugate-gradient method finds the solution to the loss function directly. The loss function is

$$2J = (y - h_k)^T W (y - h_k) + (x_0 - x_k)^T S_0 (x_0 - x_k) \quad (15.11)$$

Conjugate gradient and steepest descent both require the derivative of J , which requires H . Both find x_k that minimizes J .

15.3. Vector measurements

For small angles the transformation matrix from the reference frame to the body frame is given by

$$M = E - \theta^\times \quad (15.12)$$

where E is a 3-by-3 identity matrix and $\theta^\times$ is the skew-symmetric matrix

$$\theta^\times = \begin{bmatrix} 0 & -\theta_z & \theta_y \\ \theta_z & 0 & -\theta_x \\ -\theta_y & \theta_x & 0 \end{bmatrix} \quad (15.13)$$

The angle definitions are shown in Fig. 15.1.

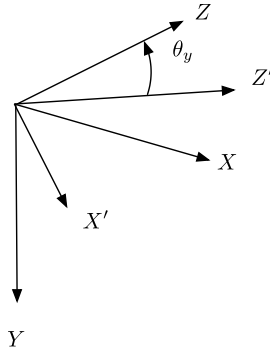


Figure 15.1 Small angles.

Assume that the transformation matrix from the body frame to the sensor frame is m_S . Then, a vector in the reference frame becomes

$$u_M = M_S(E - \theta^\times)u \quad (15.14)$$

$$u_M = M_S u - M_S \theta^\times u \quad (15.15)$$

$$u_M - M_S u = M_S \gamma^\times \theta \quad (15.16)$$

$$z = h\theta \quad (15.17)$$

where E is the identify matrix and M_S transforms to the sensor frame. This is the form of the standard linear measurement equation for a Kalman-filter formulation. h is a 3-by-3 matrix, however, the inverse of h is singular; hence it is not possible to estimate three components of θ from any set of measurements, z , which are derived from a single vector unless h (and therefore u) is time varying. If multiple, independent vectors measurements are available, the three components of θ can be estimated.

Vector measurements include star measurements, Sun measurements, planet measurements, and magnetic-field measurements. All can be combined using this formulation.

15.4. Disturbance estimation

It is often advantageous to estimate disturbance torques on a spacecraft. A simple example is a momentum bias geosynchronous satellite that is Earth-pointing. The equations of motion for roll/yaw become

$$-\frac{T_z}{h_w} = \dot{\theta}_x - \omega_o \theta_z \quad (15.18)$$

$$\frac{T_x}{h_w} = \dot{\theta}_z + \omega_o \theta_x \quad (15.19)$$

where h_w is the momentum wheel momentum, T_x is the roll torque, T_z is the yaw torque, and ω_o is orbit rate. A proportional control could be designed using these equations. The state-space matrices are

$$A = \begin{bmatrix} 0 & \omega_o \\ -\omega_o & 0 \end{bmatrix} \quad (15.20)$$

$$B = \begin{bmatrix} 0 & -\frac{1}{h_w} \\ \frac{1}{h_w} & 0 \end{bmatrix} \quad (15.21)$$

$$C = \begin{bmatrix} 1 & 0 \end{bmatrix} \quad (15.22)$$

$$D = \begin{bmatrix} 0 & 0 \end{bmatrix} \quad (15.23)$$

$$u = \begin{bmatrix} T_x \\ T_z \end{bmatrix} \quad (15.24)$$

In this case, we only have a roll-angle measurement. Note that the torques crosscouple so that the yaw torque drives roll and the roll torque drives yaw. The disturbances on the geosynchronous spacecraft are due to solar pressure. The spacecraft has solar arrays that always point at the Sun and the bus points at the Earth. Thus there will be an inertially fixed torque on the spacecraft (which in the body frame is at orbit rate), a body-fixed torque, and harmonics of orbit rate. If we limit our estimator to the body-fixed torque, orbit rate, and twice orbit rate, the augmented A matrix is

$$A = \begin{bmatrix} 0 & \omega_o & -\frac{1}{h_w} & -\frac{1}{h_w} & -\frac{1}{h_w} & 0 & 0 \\ -\omega_o & 0 & 0 & 0 & 0 & \frac{1}{h_w} & \frac{1}{h_w} \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -\omega_o & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -2\omega_o \\ 0 & 0 & 0 & \omega_o & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 2\omega_o & 0 & 0 \end{bmatrix} \quad (15.25)$$

The first row is yaw, with roll-torque and yaw-angle inputs. The second row is roll, with roll-angle and yaw-torque inputs. The third row is the roll-bias torque. The fourth row is the roll orbit-rate disturbance. The fifth row is the roll twice per orbit disturbance torque. The sixth row is the yaw once per orbit disturbance while the seventh row is the yaw twice per orbit disturbance. The yaw-bias torque is not included since it cannot be observed from a roll measurement. The estimator is comprised of the low-order roll/yaw dynamics model augmented with a disturbance model. The disturbance model assumes that the cyclic roll and yaw disturbances have the same magnitude and are 90 deg out of phase. That is why the same differential equation for the orbit rate and twice orbit rate disturbance dynamics can produce both estimated yaw- and roll-torque outputs.

Since the idea is to produce estimates of the disturbances, the estimator is modified to include known torque inputs from the control system

$$\dot{\hat{x}} = A\hat{x} + L(\phi - H\hat{x}) + Gu_c \quad (15.26)$$

$$y = C\hat{x} \quad (15.27)$$

Using full-state feedback, with no additional control dynamics,

$$u_C = -K\hat{x} \quad (15.28)$$

where K is the gain matrix and the new estimator equation becomes

$$\dot{\hat{x}} = (A - GK)\hat{x} + L(\phi - H\hat{x}) \quad (15.29)$$

$$y = C\hat{x} \quad (15.30)$$

which means that the new estimator plant is that for the closed-loop system. The spacecraft is now expected to behave as if it had the ideal control system operating. Normally, this would be done by replacing the plant matrix with the closed-loop plant matrix.

15.5. Stellar-attitude determination

15.5.1 Introduction

Attitude determination systems measure the angular offset between a spacecraft reference frame and the desired reference frame. The simplest attitude determination systems measure the misalignment directly. For example, an Earth sensor measures the roll and pitch errors directly. Yaw can be measured directly using a magnetometer. These systems measure the attitude with respect to an Earth-oriented frame. Errors with respect to inertial frames can be measured using the Sun or stars as references. Two types of recursive attitude determination systems are:

1. Recursive with gyros;
2. Recursive without gyros.

15.5.2 Gyro-based attitude determination

Gyros are often part of an attitude determination system. Gyros measure inertial angular rates in the body frame. Rate gyros output the rate and rate-integrating gyros output the integrated rate. The latter measurements are known as angular increments. To use a gyro as an attitude reference requires that at some point in time the gyro be initialized. For example with a rate-integrating gyro, the output is set to zero, and the current orientation of the spacecraft is measured by other means. This is the attitude reference. The integrated rates are then used to determine the attitude with respect to the initialized value. Since the outputs are integrated body rates, the readout cannot be used directly unless the spacecraft does not rotate very much.

For spacecraft-attitude determination, the model used in the Kalman filter is the error model for the gyros, not the dynamics and kinematics of the spacecraft. In essence, the gyros are assumed to be a perfect attitude reference except for biases (which may drift). The gyro outputs are not considered measurements. This greatly simplifies the model in the estimator. It is rarely feasible to accurately model a spacecraft in sufficient fidelity to use such a model in the estimator. Sometimes an approximation, such as a low-pass filter, represents the spacecraft when a gyro is not available.

The Kalman filter propagates two quantities. One is the state, i.e., the attitude and gyro-bias estimates. The second is the state covariance, a measure of the uncertainty of the estimates. The covariance estimate will often reach a steady state in which case the Kalman gain (that is computed the covariance) remains fixed. Sometimes, the covariance propagation is omitted and the steady-state gains used in a fixed-gain Kalman filter.

Kalman filters that are used for stellar-attitude determination are usually extended Kalman filters or unscented Kalman filters. This means that the state and measurement matrices are functions of the state. The measurement matrix changes as different stars are used and since the state propagation is nonlinear, the state-transition matrix also changes as a function of the state.

The best representation of the attitude is the quaternion that is a four-parameter set such that

$$1 = q_0^2 + q_1^2 + q_2^2 + q_3^2 \quad (15.31)$$

Propagation of the quaternion is singularity free. The q_0 component is sometimes called the scalar component and the other three are the vector component. The cost of the set being singularity free is that more than the three minimum states are needed to represent the attitude. If we were to use the following as our state vector

$$x = \begin{bmatrix} q \\ b \end{bmatrix} \quad (15.32)$$

our corresponding covariance matrix would be 7×7 , but the quaternion covariances would not be independent. The standard approach is to use the error quaternion.

It is easiest to start with transformation matrices. The small-angle rotation matrix from the unrotated frame to the rotated frame is

$$M_\delta = \begin{bmatrix} 1 & \theta_z & -\theta_y \\ -\theta_z & 1 & \theta_x \\ \theta_y & -\theta_x & 1 \end{bmatrix} \quad (15.33)$$

This matrix can be written as

$$M_\delta = E - \theta^\times \quad (15.34)$$

where E is the identity matrix and $\theta^\times$ is the skew-symmetric matrix

$$\theta^\times = \begin{bmatrix} 0 & -\theta_z & \theta_y \\ \theta_z & 0 & -\theta_x \\ -\theta_y & \theta_x & 0 \end{bmatrix} \quad (15.35)$$

The transformation matrix from the inertial frame to the body frame (which we want to estimate) is

$$M_{ib} = \hat{M}_{ib} M_\delta \quad (15.36)$$

The equivalent quaternion expression is

$$q_{ib} = \hat{q}_{ib} \otimes q_\delta \quad (15.37)$$

The small-rotation matrix can be converted to a quaternion using the following equations

$$q_\delta = \frac{1}{2} \begin{bmatrix} 2 \\ \frac{M_\delta(3,2) - M_\delta(2,3)}{2} \\ \frac{M_\delta(1,3) - M_\delta(3,1)}{2} \\ \frac{M_\delta(2,1) - M_\delta(1,2)}{2} \end{bmatrix} \quad (15.38)$$

or

$$q_\delta = \begin{bmatrix} 1 \\ -\frac{\theta_x}{2} \\ -\frac{\theta_y}{2} \\ -\frac{\theta_z}{2} \end{bmatrix} \quad (15.39)$$

or

$$q_\delta = \begin{bmatrix} 1 \\ \delta \end{bmatrix} \quad (15.40)$$

where

$$\delta = -\frac{1}{2} \begin{bmatrix} \theta_x \\ \theta_y \\ \theta_z \end{bmatrix} \quad (15.41)$$

We then can propagate

$$x = \begin{bmatrix} \delta \\ b \end{bmatrix} \quad (15.42)$$

and maintain a 6×6 covariance matrix.

The entire algorithm is as follows. Start with the standard Kalman filter,

$$k = ph^T(hph^T + r)^{-1} \quad (15.43)$$

$$x = x + k(z - hx) \quad (15.44)$$

$$p = khph^T k^T + krk^T \quad (15.45)$$

$$x = \Phi x \quad (15.46)$$

$$p = \Phi p \Phi^T \quad (15.47)$$

The first three equations are the measurement incorporation and the last two are the state update. h will depend on the number of stars so the size of h and r will vary as different numbers of stars are used as part of the measurement. The number of stars does not change the size of the covariance matrix.

For each star the measurements are the location of the star centroid in the focal plane. Assume that we have a star with unit vector u in the inertial frame. Assume that

the sensor boresight is along the z -axis. Then, the measurement for the star is

$$u_m = M_{bs} M_{ib} u_c \quad (15.48)$$

where m_{bs} transforms from the body frame to the sensor frame, m_{ib} transforms from the inertial frame to the body frame, u_c is the catalog star unit vector, u_m is the measured unit vector, and m_{ib} is the matrix equivalent of q . Our measurement is the first two elements of u_m . We can expand m_{ib} as the product of the nominal attitude and the error. We can then write this equation as

$$u_m = M_{bs} \hat{M}_{ib} M \delta u_c \quad (15.49)$$

This expands to

$$u_m = M_{bs} \hat{M}_{ib} u_c - M_{bs} \hat{M}_{ib} \theta^\times u_c \quad (15.50)$$

so that

$$h = M_{bs} M_{ib} u_c^\times \quad (15.51)$$

and the actual measurement matrix is the upper 2×3 part of h .

The state-transition matrix, Φ that updates x must also be computed. This is used only to update p and is not used to propagate the attitude state or the biases.

The equation for the quaternion time derivative is

$$\dot{q} = \frac{1}{2} \begin{bmatrix} 0 & -\omega^T \\ \omega & -\omega^\times \end{bmatrix} q \quad (15.52)$$

If we write the equation for the true quaternion (dropping the ib subscript)

$$\dot{q} = \frac{1}{2} \Omega(\omega) q \quad (15.53)$$

and the equation for the estimate is

$$\dot{\hat{q}} = \frac{1}{2} \Omega(\hat{\omega}) \hat{q} \quad (15.54)$$

If we define a four-element rate vector as

$$\omega = \begin{bmatrix} 0 \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} \quad (15.55)$$

then the rate equations can be written as

$$\dot{q} = -\frac{1}{2}\omega \otimes q \quad (15.56)$$

and

$$\dot{\hat{q}} = -\frac{1}{2}\hat{\omega} \otimes \hat{q} \quad (15.57)$$

The delta quaternion is found from

$$\delta = q \otimes \hat{q}^T \quad (15.58)$$

and

$$q = \delta \otimes \hat{q} \quad (15.59)$$

The derivative is

$$\dot{\delta} = \dot{q} \otimes \hat{q}^{-1} + q \otimes \dot{\hat{q}}^{-1} \quad (15.60)$$

Substituting for the rates we find

$$\dot{\delta} = \frac{1}{2} [\omega \otimes q \otimes \hat{q}^{-1} + q \otimes (\hat{\omega} \otimes \hat{q})^{-1}] \quad (15.61)$$

Substituting for q we get

$$\dot{\delta} = \frac{1}{2} [\omega \otimes \delta \otimes \hat{q} \otimes \hat{q}^{-1} + \delta \otimes \hat{q} \otimes (\hat{\omega} \otimes \hat{q})^{-1}] \quad (15.62)$$

Applying the identity

$$(\hat{\omega} \otimes \hat{q})^{-1} = -\hat{q}^{-1} \otimes \hat{\omega} \quad (15.63)$$

we get

$$\dot{\delta} = \frac{1}{2} [\omega \otimes \delta - \delta \otimes \hat{\omega}] \quad (15.64)$$

The rates are related by the equation

$$\hat{\omega} = \omega + b - \hat{b} \quad (15.65)$$

Therefore

$$\dot{\delta} = \frac{1}{2} [\hat{\omega} \otimes \delta - \delta \otimes \hat{\omega}] + \frac{1}{2} (\hat{b} - b) \otimes \delta \quad (15.66)$$

If we neglect nonlinear terms, and redefine

$$\delta = \begin{bmatrix} \delta_x \\ \delta_y \\ \delta_z \end{bmatrix} \quad (15.67)$$

we can write

$$\dot{\delta} = \frac{1}{2}\hat{\omega} \times \delta + \frac{1}{2}\beta \quad (15.68)$$

where β is the bias error, just as δ is the quaternion error.

Fig. 15.2 shows how q flows through the system.

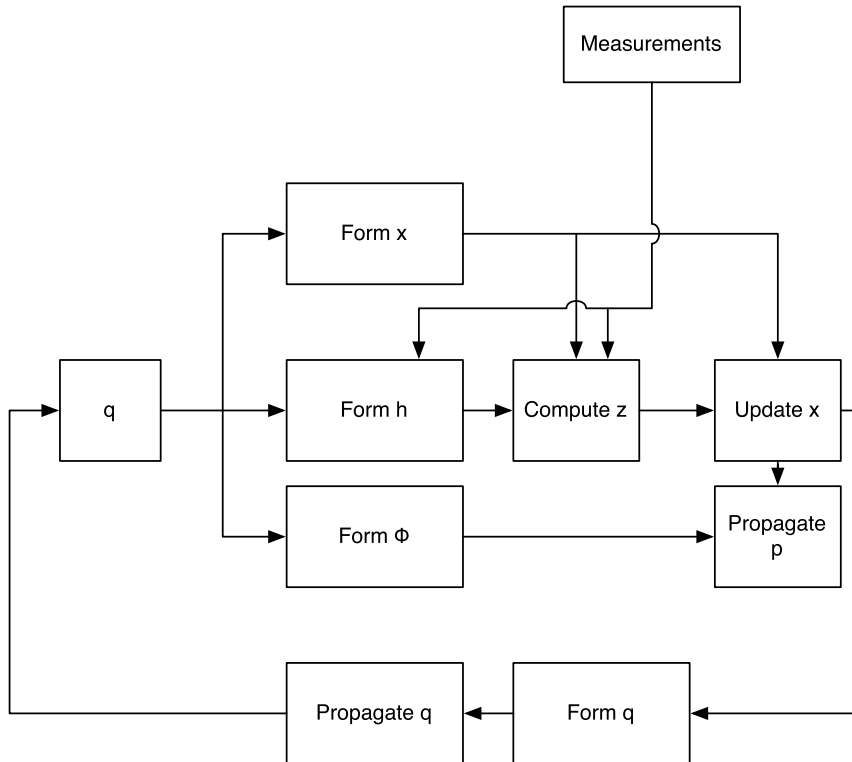


Figure 15.2 Flow of the quaternion through the Kalman filter.

15.5.3 A single-axis Kalman filter with a gyro

A simple sensing geometry is shown in the following Fig. 15.3. The sensors are used to

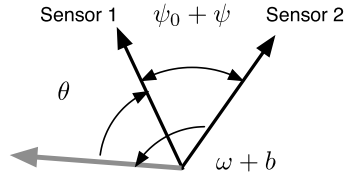


Figure 15.3 Two-sensor, single-axis sensing.

correct for the drift of the gyro that measures angular rate directly. The two sensors are separated by a known angle and an unknown bias.

The equations are

$$\dot{\theta} = \omega + b + \eta_{\theta} \quad (15.69)$$

$$b = \eta_b \quad (15.70)$$

$$\psi = \eta_{\psi} \quad (15.71)$$

$$\gamma_1 = \theta + \eta_{\gamma} \quad (15.72)$$

$$\gamma_2 = \theta + \psi + \eta_{\gamma} \quad (15.73)$$

The gyro-angle derivative is the sum of the true rate, the bias, and noise. The derivative of the gyro bias equals white noise. The derivative of the bias angle between the two angle sensors is also white noise. The two measurements are the sum of the true angle and noise. However, the second measurement includes the unknown bias.

This set of equations is linear time invariant and can be implemented using a standard Kalman filter. The results are shown in Example 15.1. The attitude estimate tracks the true attitude very well. The bias estimate converges quickly but the estimate of the angle bias between the two sensors is slow to converge.

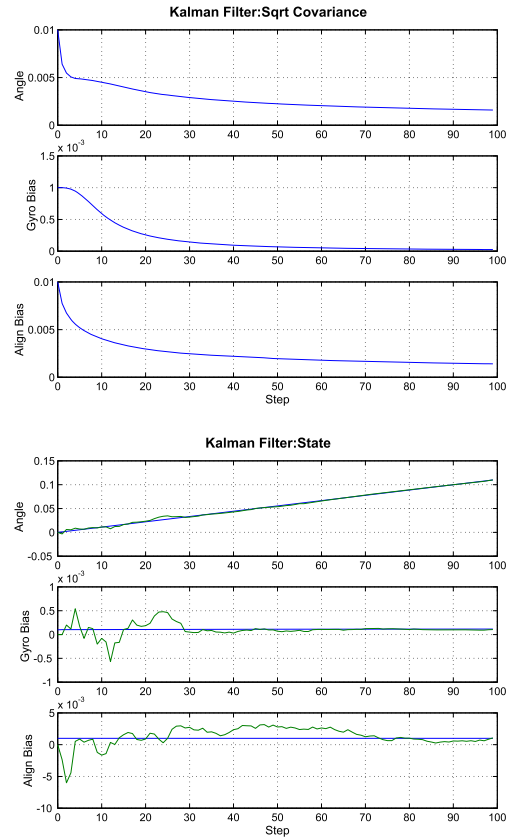
These results are reflected in the covariance square roots. The covariance plots show the square roots of the diagonal covariance elements and are representative of the expected error in the estimates. The angle covariance decreases and begins to converge. The covariance tells us that the attitude estimate will track the true attitude, despite the presence of the gyro bias. The gyro-bias estimate reflects this as it goes to zero. However, the alignment bias does not drop as quickly, meaning that we expect that there will be a bias in the estimate.

The accuracy of the estimates will depend on the relative measurement and state noise. In this demo, the noise and measurement covariances are derived directly from the noise numbers used in the simulation so, given the numbers in the simulation, these are the best results that can be obtained.

```

1 a = [0 1 0; 0 0 0; 0 0 0];
2 b = [1; 0; 0];
3 c = [1 0 0; 1 0 1];
4 dT = 1;
5
6 [a, b] = C2DZOH( a, b, dT );
7
8 nSim = 100;
9
10 xPlot = zeros(3,nSim);
11 ePlot = zeros(3,nSim);
12 pPlot = zeros(3,nSim);
13
14 omega = 0.001;
15
16 gyroBias = 1e-6;
17 alignmentBias = 1e-7;
18 measurement = 1e-2;
19 angle = 1e-5;
20
21 x = [0; omega/10; 0.001];
22 xEst = [0; 0; 0];
23 r = measurement^2*eye(2);
24 q = diag([angle^2, gyroBias^2, alignmentBias^2]);
25 p = diag([.01;.001;.01].^2);
26 z = zeros(3,1);
27
28 for k = 1:nSim
29     y = c*x + measurement*randn
30         (2,1);
31     xPlot(:,k) = x;
32     ePlot(:,k) = xEst;
33     pPlot(:,k) = sqrt(diag(p));
34     [xEst, p, g, z, pY] = KFilter( r, a, q, c, xEst,
35         y, p, [], b*omega );
36     x = a*x + b*omega + [angle;
37         gyroBias; alignmentBias].*randn(3,1);
38 end
39 n = 0:(nSim-1);
40 yLbl = strvcat('Angle','Gyro_Bias','Align_Bias');
41 Plot2D(n,[xPlot;ePlot],'Step',yLbl,'Kalman_Filter
42 State','lin',[1:4];'[2:5]';'[3:6]')
43 Plot2D(n,pPlot,'Step',yLbl,'Kalman_Filter:Sqrt_Covariance');
44 PrintFig(1,1,1,'KFStateAndEstimates');
45 PrintFig(1,1,2,'KFSRC');

```



Example 15.1: Kalman filter.

15.5.4 Star identification

It is necessary to determine which stars are visible in the field-of-view. A complete attitude determination system is illustrated Fig. 15.4.

Star identification is a major problem with any system that uses stars for attitude or orbit determination. Star cameras are used on manned spacecraft and ships for attitude determination. In these applications, they rely on the ability of a crew member to identify the stars that he or she sees and then to inform the computer of the stars' identities. Very accurate and reliable attitude determination is possible. This kind of man-in-the-loop system relies on the ability of people to rapidly identify patterns.

A practical star-identification system must deal with the issues in Table 15.1.

More details about star identification are given in Appendix K.

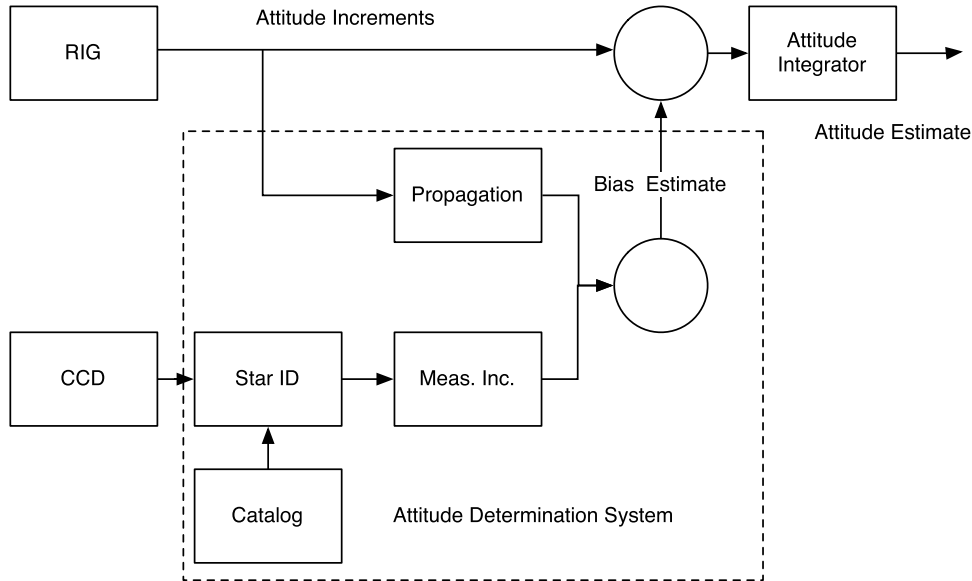


Figure 15.4 Stellar attitude determination system.

Table 15.1 Star identification.

Issue	Description
Sensor calibration	The sensing elements of the star camera may not have the same response to starlight. Thus if a star image is measured by pixel k , the resulting intensity may not be the same as when it is measured by pixel j . While the sensor can be calibrated, and a calibration map stored as part of the algorithm, the calibration may change with time.
Sensor noise	Pixels may give finite output (even after calibration) when there is no star image on the pixel.
Sensor resolution	The number of pixels are limited. Resolution usually is improved by smearing the image over several pixels. Resolution improvements of up to 100 are possible. The problem with finite resolution is that a measured dot product may agree with many catalog dot products. Thus it is necessary to look at patterns.
Other objects	The Sun, planets, moons, other spacecraft, and debris may appear in the field of view. Other objects may completely obscure the star image or appear as false stars. The latter is a more serious problem. The former is solved either by running gyros all the time or having a second-star tracker.
Stray light	Even the slightest Sun reflection that enters the camera will overwhelm the star images. Baffles are used to minimize this problem.
Speed	As an example, the Hipparcos catalog has over 100 000 stars. If the entire catalog is used, in a lost-in-space condition, comparisons need to be made with all of those stars. That can take some time.

15.6. Kalman filter with roll, pitch, and yaw and a gyro

For this Kalman filter we need roll, pitch, and yaw. These can be obtained by an Earth sensor and magnetometer. The first step to computing these angles is to compute the local vertical/local horizontal (LVLH) quaternion

$$q_{LVLH} = f(r, v) \quad (15.74)$$

where r and v are the ECI position and velocity vector. From the magnetic-field model get the unit vector of the magnetic field in the LVLH frame.

$$u_B = q_{LVLH} \otimes B_{ECI} \quad (15.75)$$

We need the unit nadir vector in the LVLH frame

$$u_N = q_{LVLH} \otimes B_{ECI} \frac{r}{|r|} \quad (15.76)$$

These two vectors are then used to produce a quaternion from body to LVLH

$$q_\delta = f(u_B, u_N) \quad (15.77)$$

q_δ is then converted into Euler angles. This is the measurement.

The state matrix is

$$\Phi = \Delta T \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (15.78)$$

The input matrix is

$$\Gamma = \Delta T \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad (15.79)$$

The measurement matrix is

$$h = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (15.80)$$

The measurement incorporation step is

$$k = \frac{pb}{hph^T + r} \quad (15.81)$$

$$x = x + k(\gamma - hx) \quad (15.82)$$

$$p = (E(6) - kh)p \quad (15.83)$$

where p is the state covariance, r is the measurement covariance, $E(6)$ is a six-by-six identity matrix, and γ is the measurement.

The state-propagation input is the gyro angle

$$\omega = \frac{\theta - \theta_{old}}{\Delta T} \quad (15.84)$$

$$x = \Phi x + \Gamma \omega \quad (15.85)$$

$$p = \Phi p \Phi^T + Q \quad (15.86)$$

$$\theta_{old} = \theta \quad (15.87)$$

where Q is the plant covariance matrix. It contains all uncertainties in the gyro model.

15.7. Kalman filter with a quaternion measurement

The Kalman filter estimates the gyro bias and the ECI to body quaternion. It has two parts, state prediction and measurement incorporation. The gyro model in state-space form is

$$x_k = ax_{k-1} + b\omega_{k-1} \quad (15.88)$$

$$\gamma_k = hx_k \quad (15.89)$$

where

$$x_k = \begin{bmatrix} \theta_x \\ \theta_y \\ \theta_z \\ b_x \\ b_y \\ b_z \end{bmatrix} \quad (15.90)$$

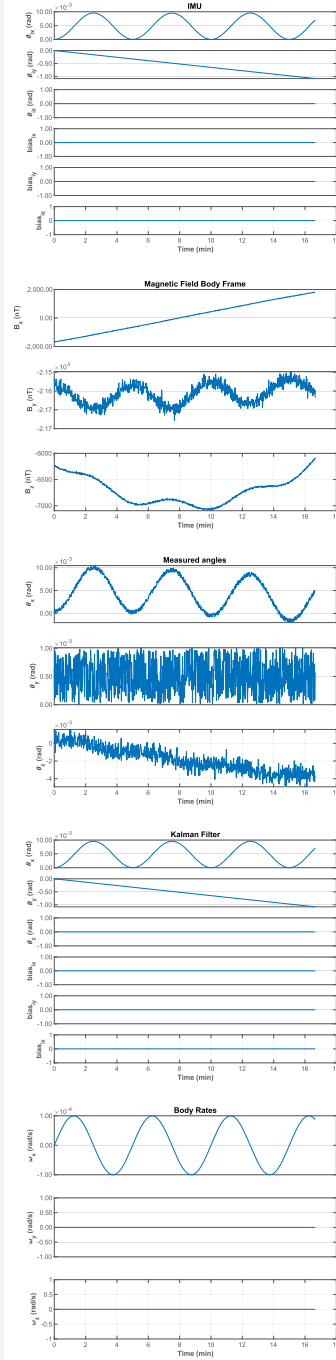
The matrices are

$$a = \begin{bmatrix} 0 & 0 & 0 & \Delta t & 0 & 0 \\ 0 & 0 & 0 & 0 & \Delta t & 0 \\ 0 & 0 & 0 & 0 & 0 & \Delta t \end{bmatrix} \quad (15.91)$$

```

1 dKF = GyroKalmanFilter;
2 dRHS = RHSGyroWithOrbit;
3 jD0 = Date2JD([2041 7 5]);
4 bias = [0;0;0];
5 iMUAngle = [0;0;0];
6 e1 = [7000 0 0 0 0];
7 q = [1;0;0;0];
8 n = 1000;
9 dT = 1;
10 bNoise = 1e-8;
11 measNoise = 1e-3;
12 dOmega = [0.0001;0;0];
13 omega0 = 2*pi/300;
14
15 % Initialize the Kalman Filter
16 dKF.r = measNoise*2*eye(3);
17 dKF.q = [diag(dRHS.ang1Sig.^2) zeros(3,3)
18         ;... zeros(3,3) diag(dRHS.
19         b1Sig.^2)];
20
21 %% Simulate
22 [r,v] = EI2RV(e1);
23 orbRate = 2*pi/Period(e1(1));
24 q = QLVH(r,v);
25 x = [r;v;q;iMUAngle;bias];
26 secInDay = 86400;
27 dKF.omega = [0;-orbRate;0];
28
29 % Allocate memory
30 xP = zeros(length(x)+15,n);
31
32 % Create a time vector
33 t = (0:n-1)*dT;
34 jD = jD0 + t/secInDay;
35
36 % Simulation loop
37 for k = 1:n
38
39     % Body rate
40     dRHS.omega = [0;-orbRate;0] + dOmega*sin(omega0
41         *t(k));
42
43     % Convenient to use local variables
44     r = x(1:3);
45     v = x(4:6);
46     qECIToBody = x(7:10);
47     iMUAngle = x(11:13);
48
49     % Sensor measurements
50     bMeas = QForm(qECIToBody,BDipole(r,jD(k))) +
51         bNoise*randn(3,1);
52     nadirMeas = Unit(-QForm(qECIToBody,r));
53
54     % Angles with respect to LVLH
55     meas = AngleDetermination(r,v,nadirMeas,bMeas,
56         jD(k)) + measNoise*rand(3,1);
57
58     % Kalman Filter
59     dKF = GyroKalmanFilter(meas,iMUAngle,dKF);
60
61     % Do one simulation step
62     xP(:,k) = [x;bMeas*1e9;meas;dKF.x;dRHS.omega];
63     x = RK4(@RHSGyroWithOrbit,x,dT,0,dRHS);
64
65 % Put plot labels in a cell array
66 yL = {'x_u(km)' 'x_v(km)' 'x_w(km)' ...
67     'v_x_u(km/s)' 'v_y_u(km/s)' 'v_z_u(km/s)' ...
68     'q_x' 'q_y' 'q_z' ...
69     '\theta_x(rad)' '\theta_y(rad)' '\theta_z(rad)' ...
70     'bias_{ix}' 'bias_{iy}' 'bias_{iz}' ...
71     'B_x_u(nT)' 'B_y_u(nT)' 'B_z_u(nT)' ...
72     '\theta_x_u(rad)' '\theta_y_u(rad)' '\theta_z_u(rad)' ...
73     '\theta_x_v(rad)' '\theta_y_v(rad)' '\theta_z_v(rad)' ...
74     'bias_{ix}' 'bias_{iy}' 'bias_{iz}' ...
75     '\omega_x_u(rad/s)' '\omega_y_u(rad/s)' '\omega_z_u(rad/s)'};
76
77 % Generate the plots.
78 tL = {'Position' 'Velocity' 'Quaternion' 'IMU'
79     ...
80     'Magnetic_Field_Body_Frame' 'Measured_
81     angles' ...
82     'Kalman_Filter' 'Body_Rates'};
83 kL = {1:3 4:6 7:10 11:16 17:19 20:22 23:28
84     29:31};
85
86 for j = 1:length(tL)
87     k = kL{j};
88     TimeHistory(t,xP(k,:),yL(k),tL{j});
89 end
90
91 Figui

```



Example 15.2: Kalman filter with angle measurements and a gyro.

$$b = \begin{bmatrix} \Delta t & 0 & 0 \\ 0 & \Delta t & 0 \\ 0 & 0 & \Delta t \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad (15.92)$$

$$h = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \quad (15.93)$$

The Kalman-filter measurement incorporation step is

$$k = \frac{ph^T}{hph^T + r} \quad (15.94)$$

$$x_e = k(\gamma - hx_e) \quad (15.95)$$

$$p = (E - kh)p \quad (15.96)$$

where E is a six-by-six identity matrix, p is the covariance matrix, r is the measurement covariance matrix, and h is the measurement matrix. The measurement is γ , which is the delta quaternion

$$\delta q = (q_m^T \otimes q_e)^T \quad (15.97)$$

where q_m is the measured quaternion and q_e is the current estimate of the quaternion. The input to the measurement update are elements 2 through 4 of δq .

$$u_\omega = \frac{\omega}{|\omega|} \quad (15.98)$$

$$\theta = \frac{|\omega|\Delta t}{2} \quad (15.99)$$

$$\delta q = \begin{bmatrix} \cos \theta & u_\omega \sin \theta \\ -u_\omega \sin \theta & E \cos \theta - (u_\omega \sin \theta)^\times \end{bmatrix} q \quad (15.100)$$

$$p = apa^T + q \quad (15.101)$$

where E is a 3-by-3 identity matrix, a is the state-transition matrix, q is the model covariance matrix, and Δt is the time step. The output of the rate-integrating gyro is differenced to get rate, ω . This is where the gyro connects to the model. IMU angles are treated as a state-prediction input, not a measurement.

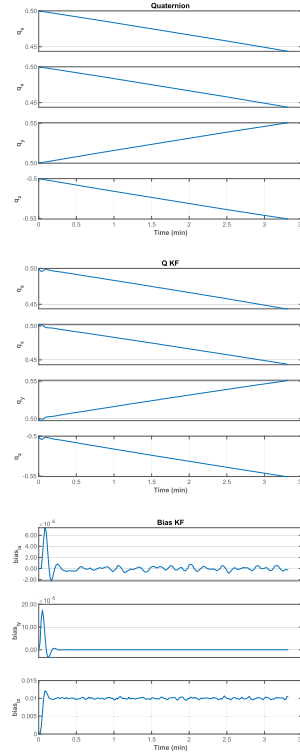
The Kalman filter produces an updated delta state. The quaternion part is incorporated as follows. Let δ be elements 2 through 4 of δq

$$n = |\delta|^2 \quad (15.102)$$

```

1 dKF = GyroKalmanFilter;
2 dRHS = RHSGyroWithOrbit;
3 jD0 = Date2JD([2041 7 5]);
4 bias = [0;0;0.01]; iMUAngle = [0;0;0];
5 e1 = [7000 0 0 0 0];
6 n = 200;
7 dT = 1;
8 bNoise = 1e-8;
9 measNoise = 1e-3;
10 dOmega = [0;0;0.0;0];
11 omega0 = 2*pi/300;
12
13 % Initialize the Kalman Filter
14 dKF.r = measNoise*2*eye(3);
15 dKF.p = 0.1*dKF.q;
16
17 %% Simulate
18 [r,v] = EIDRV(e1);
19 orbRate = 2*pi/Period(e1(1));
20 q = QVLH(r,v);
21 x = [r;v;q;iMUAngle;bias];
22 secInDay = 86400;
23
24 % Allocate memory for plotting to speed things up
25 xP = zeros(length(x)+26,n);
26
27 % Create a time vector
28 t = (0:n-1)*dT;
29 jD = jD0 + t/secInDay;
30
31 for k = 1:n
32 % Body rate
33 dRHS.omega = [0;-orbRate;0] + dOmega*sin(omega0*t(k));
34
35 r = x(1:3);
36 v = x(4:6);
37 qECIToBody = x(7:10);
38 iMUAngle = x(11:13);
39
40 % Sensor measurements
41 bECI = BDipole(r,jD(k));
42 bMeas = QForm(qECIToBody,bECI)...
43 + bNoise*randn(3,1);
44 nadirMeas = Unit(-QForm(qECIToBody,r));
45
46 % This computes angles with respect to LVLH
47 meas = AngleDetermination(r,v,nadirMeas,bMeas,jD(k))...
48 + measNoise*rand(3,1);
49
50 qMeas = TwoUtoQ(Unit(-r),bECI,nadirMeas,bMeas);
51
52 % Kalman Filter
53 dKF.rECI = r;
54 dKF.vECI = v;
55 if (k == 1)
56 dKF.qECIToBody = qMeas;
57 end
58 dKF = GyroKalmanFilter(qMeas,iMUAngle,dKF);
59
60 % Do one simulation step
61 xP(:,k) = [x;bMeas*1e9;meas;dKF.x;dRHS.omega;qMeas;dKF.qECIToBody;dKF.b];
62 x = RK4(@RHSGyroWithOrbit,x,dT,0,dRHS);
63 end
64
65 % Put plot labels in a cell array
66 yL = {'x,(km)' 'x,(km)' 'x,(km)' ...
67 'v_x,(km/s)' 'v_y,(km/s)' 'v_z,(km/s)', ...
68 'q_x' 'q_y' 'q_z' ...
69 '\theta_{ix}(rad)' '\theta_{iy}(rad)' '\theta_{iz}({
70 bias_{ix}' bias_{iy}' bias_{iz}' ...
71 'B_x,(nT)' 'B_y,(nT)' 'B_z,(nT)' ...
72 '\theta_{x_u}(rad)' '\theta_{y_u}(rad)' '\theta_{z_u}(
73 '\theta_{x_u}(rad)' '\theta_{y_u}(rad)' '\theta_{z_u}(
74 bias_{ix}' bias_{iy}' bias_{iz}' ...
75 '\omega_{x_u}(rad/s)' '\omega_{y_u}(rad/s)' '\omega_{z_u}
76 'q_x' 'q_y' 'q_z' 'q_s' 'q_x' 'q_y' 'q_z'
77 bias_{ix}' bias_{iy}' bias_{iz}'];
78
79 % Generate the plots. Notice how we index a cell
80 tL = {'Position' 'Velocity' 'Quaternion' 'IMU'
81 'Magnetic_Field_Body_Frame' 'Measured_
82 angles' ...
83 'Kalman_Filter' 'Body_Rates' 'Q_Measured'
84 'Q_KF' 'Bias_KF'};
85
86 kL = {1:3 4:6 7:10 11:16 17:19 20:22 23:28 29:31
87 32:35 36:39 40:42};
88 for j = 1:length(tL)
89 k = kL{j};
90 TimeHistory(t,xP(k,:),yL(k),tL{j});
91 end

```



Example 15.3: Kalman-filter performance. The quaternion is tracked accurately and the bias is estimated.

$$\delta q_1 = \frac{16 - n}{16 + n} \quad (15.103)$$

$$\rho = \frac{1}{4}(1 + \delta q_1)\delta \quad (15.104)$$

$$\delta_q = \begin{bmatrix} \delta q_1 \\ \rho \end{bmatrix} \quad (15.105)$$

$$q = q \otimes \delta_q \quad (15.106)$$

δq must be normalized prior to the quaternion multiplication.

Example 15.3 shows the Kalman-filter response. It tracks the true quaternion very well and estimates the gyro bias accurately.